

# PSMDSRC103

## ASSESSMENT 3.2

### RAFFY BASAL

#### AREA 1: UNDERSTANDING OBJECTS

##### 1. Understanding Objects

Define a class called Address that has two attributes: number and street name. Make sure you have an init method that initializes the object appropriately. You do not need to define any other methods.

```
Class Address:
def __init__(self, number, street_name):
    self.number = number
    self.street_name = street_name
```

##### 2. Consider the following code:

(a) What does the code print out? Guess first, and then create a Python file and run it.

**GUESS:** It will return an error, because the '6:30' is not enclosed to () bracket, which is the value of time, but if these would proceed it will print '5:30' since it was the object assigned to the clock.

**PYTHON:** It is possible to have the value of a string to be not enclosed in a bracket, but Python is not supporting an angled ('), only straight quotes that's why it returns an error.

```
1 class Clock(object):
2     def __init__(self, time):
3         self.time = time
4     def print_time(self):
5         time = '6:30'
6         print self.time
7 clock = Clock('5:30')
8 clock.print_time()
```

0.0s

Cell In[1], line 5  
time = '6:30'  
^  
SyntaxError: invalid character "'" (U+2019)

(b) Why does the code print this?

If we correct the code, it prints the Instance time '5:30' time since it was the object assigned in the clock time. It doesn't affect the local variable of a self.time since the variable time inside print\_time is local to that method.

```
1 class Clock(object):
2     def __init__(self, time):
3         self.time = time
4
5     def print_time(self):
6         time = '6:30'
7         print("Instance time:", self.time)
8         print("Local time:", time)
9
10 clock = Clock('5:30')
11 clock.print_time()
```

0.0s

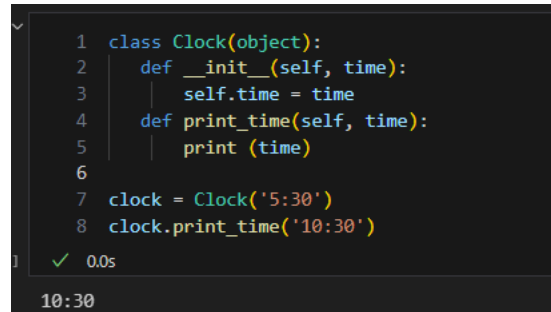
Instance time: 5:30  
Local time: 6:30

3. Consider the following code:

(a) What does the code print out? Guess first, and then create a Python file and run it.

**GUESS:** It will print the time '10:30', although there is an assigned object on the clock instance, but there is also assign on the clock.print\_time instance.

**PYTHON:** It print out the '10:30' value as assigned when we call the clock.print\_time instance.



```
1 class Clock(object):
2     def __init__(self, time):
3         self.time = time
4     def print_time(self, time):
5         print (time)
6
7 clock = Clock('5:30')
8 clock.print_time('10:30')
```

✓ 0.0s

10:30

b) What does this tell you about giving parameters the same name as object attributes?

If you give a method parameter the same name as an object attribute (instance variable), the method parameter will **hide** or **override** the object attribute inside that method. This means that, in the method, you will only be able to access the value of the parameter, not the object attribute, unless you explicitly refer to the attribute using self. If a parameter and an object attribute have the same name, the parameter takes priority within that method.

4. Consider the following code:

a. What does the code print out? Guess first, and then create a Python file and run it.

**GUESS:** This will return an error because of the line 5 the print self.time is not enclosed by a () bracket, but if this will proceed the print out of this code will be '10:30' because it is the value assigned on the local variable 'boston\_clock' time in the given code. It derives from paris\_clock.time which is '10:30', and paris\_clock are equally assigned to boston\_clock. Although, there is a given object attribute for boston, it will be override by a new parameter which is derived from the paris\_clock.

**PYTHON:** As I guess, the new parameter override's the assigned object because parameters is prioritized within the method. And the assignment of paris\_clock = boston\_clock only means that both variables refer to the same object.

- b. Why does it print what it does? (Are boston clock and paris clock different objects? Why or why not?)

It printed the '10:30', because paris\_clock and boston\_clock are assigned equally, thus the value of assigned to paris\_clock will be also the value of boston\_clock, unless you will create a new instance to override the code.

Boston\_clock and paris\_clock are **not different objects**. They are just two different variables pointing to the **same object** in memory. Any changes made through one will be reflected in the other because they reference the same object.

## AREA 2: DESIGNING YOUR OWN CLASS

For this exercise, you will be coding your very first class, a Queue class. Queues are a fundamental computer science data structure. A queue is basically like a line at Disneyland - you can add elements to a queue, and they maintain a specific order. When you want to get something off the end of a queue, you get the item that has been in there the longest (this is known as 'first-in-first-out', or FIFO). You can read up on queues at Wikipedia if you'd like to learn more.

Create a new file called queue.py to make your Queue class. In your Queue class, you will need three methods:

- init : to initialize your Queue (think: how will you store the queue's elements? You'll need to initialize an appropriate object attribute in this method)
- insert: inserts one element in your Queue
- remove: removes one element from your Queue and returns it. If the queue is empty, return a message that says it is empty (without throwing an error that halts your code).

```
tempCodeRunnerFile.py > ...
1 class Queue:
2     def __init__(self):
3         # Initialize an empty list to store the queue's elements
4         self.queue = []
5
6     def insert(self, element):
7         # Insert element at the end of the queue
8         self.queue.append(element)
9
10    def remove(self):
11        # Remove and return the first element from the queue
12        if len(self.queue) > 0:
13            return self.queue.pop(0) #removing the first item until such that will return empty to follow FIFO
14        else:
15            return "The queue is empty."
16
17 # Testing the Queue class (no file import)(security check to ensure that the program is run directly)
18 if __name__ == "__main__":
19     queue = Queue()
20
21     # Inserting elements
22     queue.insert(5) # 1st element, also first to be removed
23     queue.insert(6)
24     queue.insert(7)
25
26     # Removing elements and printing results
27     print(queue.remove()) # Should print 5
28     print(queue.remove()) # Should print 6 because of 'pop(0)' (FIFO)
29     print(queue.remove()) # Should print 7
30     print(queue.remove()) # Should print "The queue is empty."
```

## AREA 3: OOP PARADIGMS

For this exercise, we want you to describe a generic superclass and at least three subclasses of that superclass, listing at least two attributes that each class would have. Remember what classes will inherit (will subclasses inherit attributes from their parent class? Will parent classes inherit from their subclasses? Will subclasses that share a parent inherit from one another? Be sure you're clear on this before continuing with this exercise.) It's easiest to simply describe a real-world object in this manner. An example of what we're looking for would be to describe a generic Shoe class and some specific subclasses with attributes that they might have, as shown here:

```
class MensShirt:
    # Attributes: self.size, self.color
#SUBCLASS 1
class TShirt(MensShirt): # Inherits from MensShirt
    # Attributes: self.fabric, self.sleeve_length

#SUBCLASS 2
class FormalShirt(MensShirt): # Inherits from MensShirt
    # Attributes: self.collar_type, self.cuff_type

#SUBCLASS 3
class PoloShirt(MensShirt): # Inherits from MensShirt
    # Attributes: self.button_count, self.logo_present
```

## Conclusion & Analysis

Object attributes can be overridden with new parameters because of how variable scope works in programming. When you define a parameter in a method with the same name as an object attribute, the parameter takes priority within that method. This allows you to use a temporary value for that method without changing the actual object attribute.

For example, in the `boston_clock` and `paris_clock`. We have defined the value of `boston_clock`, but since we also define `paris_clock = boston_clock`, this is a new parameter that is the value given to the `paris_clock` is the same value on `boston_clock` when called.

Importance of naming conventions and the `if __name__ == "__main__":` statement in Python as a security check if we are the program is will run directly or an import file will be used. Using unique names for parameters and instance variables is important to avoid confusion. Additionally, we looked at how methods like `insert` and `remove` in the `Queue` class help manage data without exposing the details of how the class works. Also, the use of `pop(0)` to removed the first data provided from the code and `display's` the next data, and so on.

In the exploration of object-oriented programming, we learned how to create and organize classes using examples like shoes, men's shirts and others. We discovered that a superclass can represent general characteristics, while subclasses can inherit these attributes and also have specific features of their own. This inheritance approach allows us to reuse code and keep it organized, making it easier to manage and understand. By using attributes and methods, we can reflect real-life relationships in our programs, which helps in creating clear and structured code. This structure not only promotes efficiency but also enhances readability, making it simpler for developers to work with and modify their code over time.